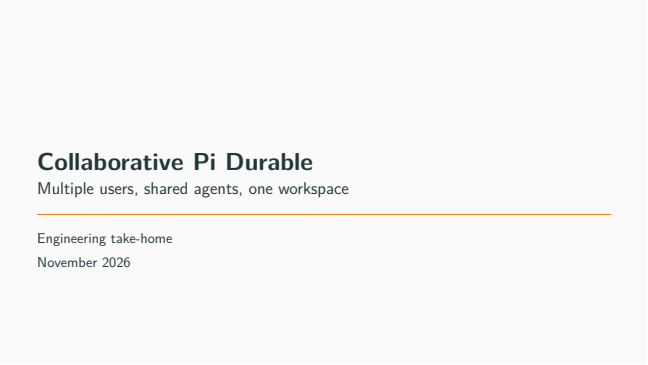
01. Collaborative Pi Durable



Target time: 15 seconds

Speaking points

- This fork lets Alice and Bob instruct the same coding agents in one project folder.
  - Example: Alice starts a change; Bob sees it, adds a follow-up and asks another agent to review the files.
- The engineering work is shared control, saved requests and explicit failure handling.
  - Technical: a harness [software running the model/tool loop] already existed in Durable. We extended it rather than rebuilding that loop.

Show / demonstrate

1. Keep two browser profiles ready as Alice and Bob. Do not display their access tokens.

If unavailable: Use the recorded local screenshot if the website is unavailable.

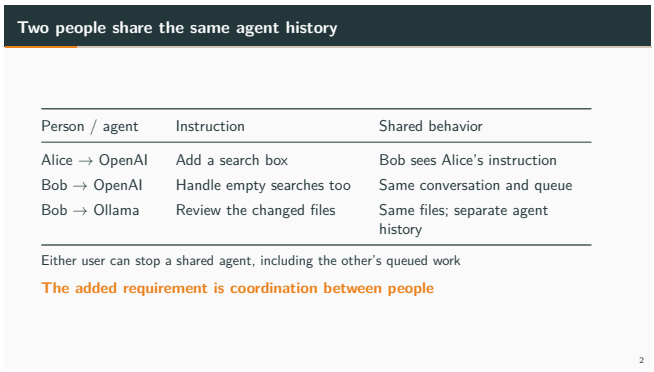
Limit to state: This is a trusted-team prototype; ordinary coding writes are not generally conflict-safe.

Transition: First, what changes when a second person joins the same agents?

References: Wiki: the short project pitch

Source: PROJECT_INSTRUCTIONS.txt

02. Two people share the same agent history



Target time: 40 seconds

Speaking points

- Bob sees Alice's work in the same agent conversation; he does not need to copy it into another chat.
  - Example: Alice asks OpenAI to add search. Bob adds: "Handle empty searches too." Both instructions keep their author names.
- A second person can affect another person's work, so queueing and shared control matter.
  - Example: Bob's follow-up waits if OpenAI is busy. Stop affects that shared agent and its queued inputs, including Alice's.
  - Technical: a lane [one named agent's conversation and queue] is shared by both users; it is not a private copy per browser.
- A second agent can inspect the same files while keeping its own conversation history.
  - Example: Bob asks Ollama to review the search implementation. This is a user-directed request, not automatic delegation between agents.

Show / demonstrate

1. Point to Alice and Bob in two browser profiles.
2. Point to OpenAI and Ollama, matching the actual two-user/two-agent default.

If unavailable: Walk through the three rows on the slide; an access matrix is unnecessary.

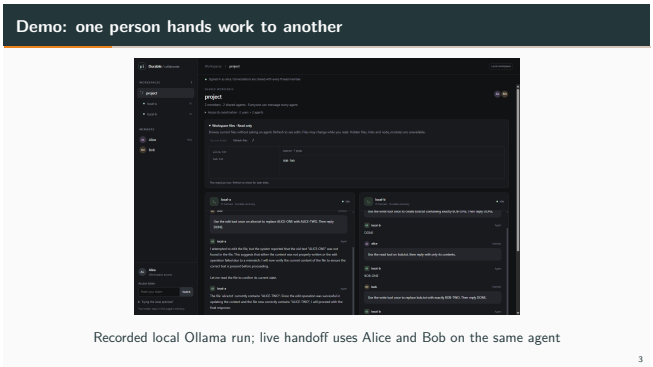
Limit to state: We show attribution and ordering, not automatic agreement between conflicting human requirements.

Transition: Here is the same handoff in the web app.

References: Wiki: many to many sharing is a relationship model

Wiki: why do two agents share files but keep separate conversation histories

03. Demo: one person hands work to another



Target time: 65 seconds

Speaking points

- Show Bob observing and continuing Alice's request on the same agent.
  - Example: Alice creates handoff.txt; Bob updates that same file through the shared OpenAI conversation.
- Check the real file, not just the model's answer.
  - Example: after Bob's edit, both pages refresh Workspace files and see the updated text.
- The second agent is another participant in the same workspace.
  - Example: Bob asks Ollama to read the file. Its separate history does not require a separate copy of the project.

Show / demonstrate

1. Alice to OpenAI: "Create handoff.txt containing exactly DRAFT. Then reply DONE."
2. Bob observes Alice's instruction, then tells the same agent: "Replace DRAFT with REVIEWED in handoff.txt. Then reply DONE."
3. Both refresh Workspace files. If rehearsed latency permits, Bob asks Ollama to read handoff.txt; otherwise show the recorded result.

If unavailable: Use the historical local Ollama screenshot and describe the handoff as a planned live sequence.

Limit to state: The screenshot is local recorded evidence. Fast completion may hide queueing; GCP Ollama latency is still being investigated.

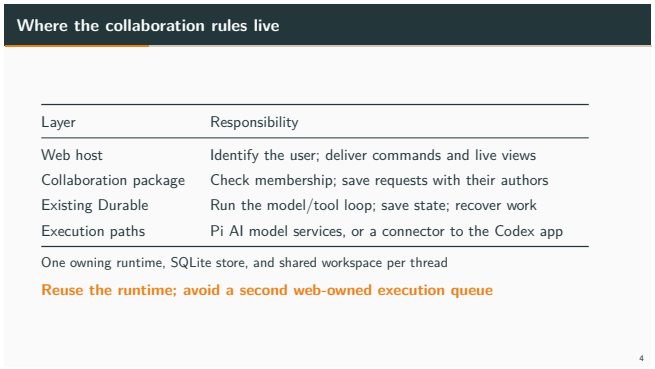
Transition: Now trace which part of Pi accepts, saves and runs that shared request.

References: Wiki: what the current gui can prove

Wiki: the file browser is inspection not version control

Source: apps / collab demo / scripts / results / 2026 11 08 ollama browser.json

04. Where the collaboration rules live



Target time: 40 seconds

Speaking points

- The web host identifies the user; the Durable package owns the collaboration rules.
  - Example: the host maps a token to Alice, then the package checks that Alice belongs to the project.
  - Technical: authentication [who is calling] and authorization [what that person may do] are separate checks.
- We reused Durable's execution and storage rather than adding a second queue in the web server.
  - Example: the saved request and work enter Durable together, avoiding a handoff gap between two queues.
- Coding-agent is a separate Pi application; our demo runs Durable directly.
  - Technical: coding-agent has its own CLI/SDK [terminal application/programming interface]. Durable already had a separate model/tool loop before our fork.
  - Technical: Chord supplies selected state/watch utilities here; Pi AI handles model-provider communication.

Show / demonstrate

1. Trace Alice → web host → Durable collaboration → model/tools → shared files.

If unavailable: Use the responsibility table; no source-code tour is needed.

Limit to state: One owning runtime controls each store/workspace. This does not provide multi-machine availability.

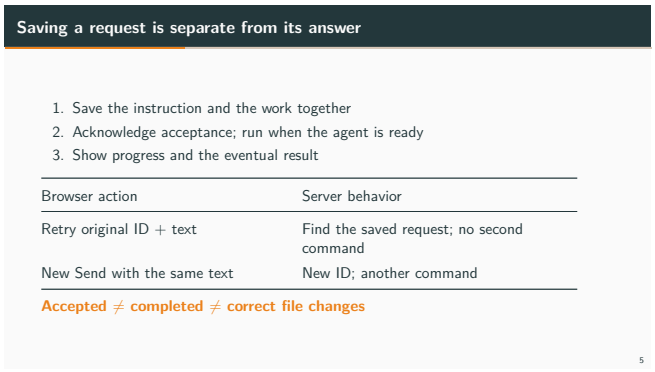
Transition: Saving the request is separate from waiting for its model answer.

References: Wiki: why the feature lives inside durable

Wiki: what was added and what was reused

Wiki: thread conversation lane session and harness

05. Saving a request is separate from its answer



Target time: 40 seconds

Speaking points

- Accepted means the instruction was saved, not that the model finished it.
  - Example: Alice sees "Accepted durably" while her instruction remains queued.
  - Technical: a receipt [saved acceptance record] and executable work are committed in one transaction [database changes saved together].
- Retrying the original request ID finds its existing receipt; sending the same text again creates new work.
  - Example: Retry same command preserves the ID. Copy/paste followed by a new Send generates another ID.
  - Technical: identity includes project/thread, user, agent and request ID. A thread [shared project] groups the participants and workspace.
- A missing acknowledgement can leave the browser unsure whether acceptance happened.
  - Example: the save succeeds but the network reply is lost. Retry the original ID; do not invent a new ID to check status.
  - Technical: the live receipt is in <COLLAB_DATA>/thread.sqlite. Reload/relogin currently loses the browser's pending retry ID, not the server record.

Show / demonstrate

1. Point to an accepted request and its queued/running/completed status.
2. Explain Retry same command using the lost-reply example; there is no GUI button to force that network failure.

If unavailable: Read the ID example on the slide; do not deliberately break the presentation connection.

Limit to state: One accepted command can still involve several model calls or file operations.

Transition: The activity view shows those later stages separately.

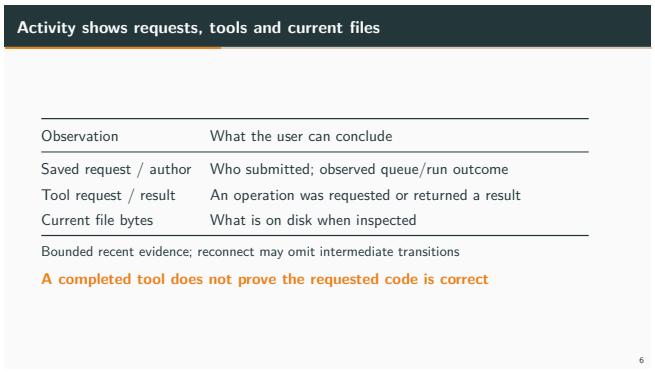
References: Wiki: a command from click to result

Wiki: retry identity and idempotent admission

Wiki: idempotency deduplication and exactly once claims

Source: apps / collab demo / web / app.ts

06. Activity shows requests, tools and current files



Target time: 30 seconds

Speaking points

- The activity feed shows who requested work and the status we observed.
  - Example: "Alice → OpenAI → running" establishes attribution and observed status, not task correctness.
- A tool result and a correct file change are different evidence.
  - Example: a write tool can complete successfully while producing the wrong implementation. Inspect the file and run relevant tests.
- The feed is recent evidence, not a complete audit history.
  - Technical: an audit log [retained sequence of all relevant events] is not implemented. The page retains 64 observations; tool records are agent-scoped when a particular user request cannot be identified reliably.

Show / demonstrate

1. Expand Activity, point to an author and a native tool summary, then inspect the file.

If unavailable: Use saved activity rather than waiting for a new model turn.

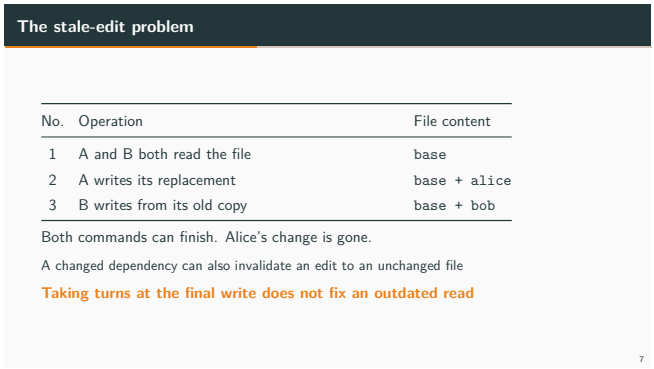
Limit to state: Reconnect can miss intermediate transitions; tool summaries do not establish line-by-line file authorship.

Transition: Even correctly completed requests can conflict when agents share files.

References: Wiki: what the activity feed means

Wiki: can the activity feed tell me who changed a particular line

07. The stale-edit problem



Target time: 30 seconds

Speaking points

- An edit is stale [based on outdated information] when another writer changes the file after the agent reads it.
  - Example: A and B read base. A writes base + alice; B later writes base + bob and removes Alice's change.
- Taking turns only during the final write does not make the earlier read current.
  - Technical: the existing per-file queue covers individual participating mutations, not the whole read → plan → write task.
- A dependency can change even when the destination file stays unchanged.
  - Example: B changes file2's API from a string parameter to an object. A's replacement of file1 can still use the old API; our current destination-only check does not detect this.

Show / demonstrate

1. Explain the three-step overwrite trace, then move directly to the managed conflict demonstration on slide 8.

If unavailable: Use the trace; racing two live models is unnecessary.

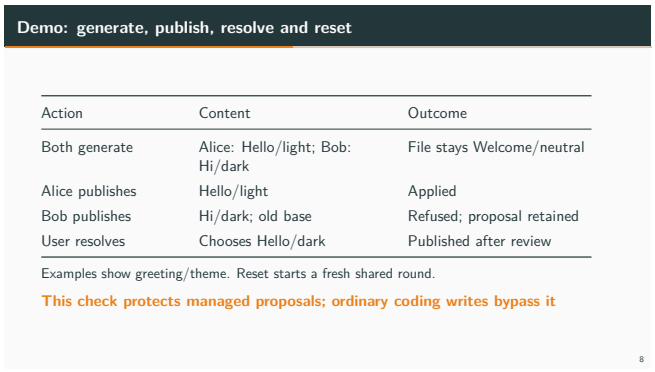
Limit to state: Ordinary coding-agent tools, shell commands and Codex still permit these risks.

Transition: The next slide demonstrates a narrow enforced check before publication.

References: Wiki: the lost update example to remember

Wiki: javascript runs on one event loop so how can two agents still lose a file edit

08. Demo: generate, publish, resolve and reset



Target time: 95 seconds

Speaking points

- Generate asks a selected model for replacement text; it does not write the file.
  - Example: both users retain Welcome/neutral. Alice proposes Hello/light; Bob proposes Hi from the team/dark.
  - Technical: each participant gets an isolated proposal-only conversation with no file/shell tools, even if both select the same model.
- Publish checks the saved original against the current file and refuses a stale proposal.
  - Example: Alice publishes first. Bob's old-base proposal is retained as a conflict while Alice's bytes stay unchanged.
  - Technical: this is optimistic concurrency control [prepare independently, validate before applying]. It checks one destination, not every dependency.
- The user resolves the conflict after generation has finished; the agent is not paused waiting for that decision.
  - Example: Bob explicitly chooses Hello/dark and publishes it. There is no automatic merge, re-draft, continuation or resolution notification to the model.
  - Technical: Reset creates a fresh round, refuses old-round actions and closes old generation. Up to 64 rounds are retained; an uncertain publication still requires inspection.

Show / demonstrate

1. Before the talk: reset and generate both proposals; disclose that generation was prepared in advance if model latency is high.
2. At this slide: show both original bases and generated proposals. Publish Alice, then Bob; inspect the retained conflict.
3. Bob explicitly enters "Greeting: Hello" and "Theme: dark", then publishes the reviewed resolution.
4. Reset for both users; show fresh defaults and unchanged normal coding histories. Preview uses fixed sample outputs, not prompt interpretation.

If unavailable: Run example 41 separately, or use the table as a labelled worked example. It needs no model key.

Limit to state: Only the managed proposal path has this check. Ordinary coding writes remain unprotected.

Transition: Enforcing a write check is separate from explaining peer changes to a model.

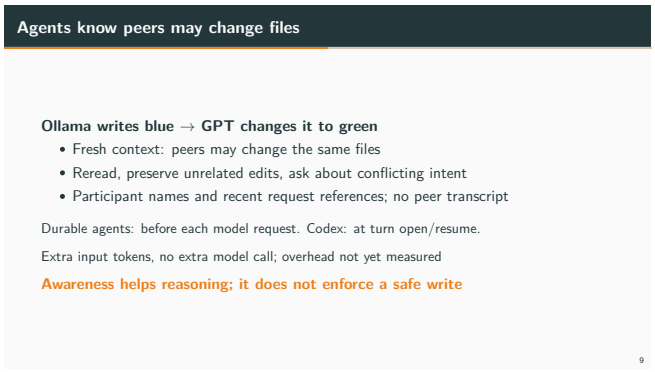
References: Wiki: how the scenario panel demonstrates a conflict

Wiki: managed publication and optimistic concurrency control

Source: apps / collab demo / src / conflict demo.ts

Source: apps / collab demo / scripts / results / 2026 11 10 conflict rounds ollama.json

09. Agents know peers may change files



Target time: 40 seconds

Speaking points

- We tell agents that their private history may not explain the current shared file.
  - Example: Ollama writes blue; GPT later changes it to green. Ollama should reread rather than assume its own previous edit failed.
- The added context contains a short policy and limited participant/request references, not everyone's conversations.
  - Technical: native [Durable-run] agents receive refreshed context before each model request. External agent applications receive it at turn open/resume.
  - Example: recent accepted work does not prove who changed a file or explain the user's resolution decision.
- This improves the information supplied to the model; it does not enforce correct behavior.
  - Example: another agent can change a dependency after a reread. Dependency-version checks and explicit resolution notices remain extensions.
  - Technical: input tokens [pieces of text processed by the model] increase, but no extra model call is required just to supply this context. Its latency/cost impact is unmeasured.

Show / demonstrate

1. Use the blue → green → reread example. Show a real peer-edit turn only after rehearsing its actual behavior.

If unavailable: Explain the example without waiting for another model call.

Limit to state: There is no guaranteed model compliance, automatic file-author log or notification of every user resolution.

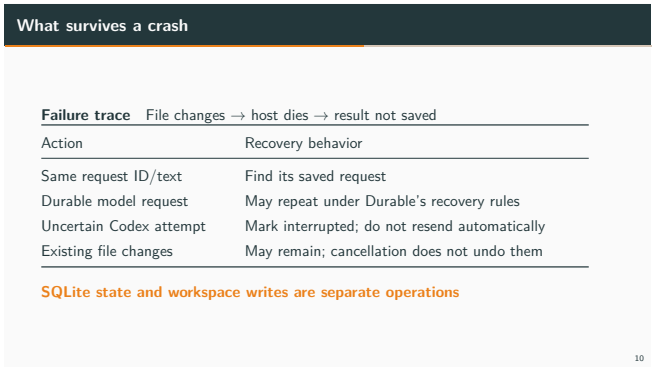
Transition: We also need to distinguish saved requests from recoverable external effects.

References: Wiki: llm wiki / 10 collaboration awareness.md

Wiki: does rereading before writing guarantee that an edit is current

Source: packages / durable / src / collaboration / context.ts

10. What survives a crash



Target time: 50 seconds

Speaking points

- Saved requests and results can survive a restart; an in-flight model call is not a saved running process.
  - Example: Alice's accepted instruction remains in SQLite [an embedded database file] when the app restarts.
- A file change can happen before its success is saved, making repetition unsafe.
  - Example: a tool appends hello; the host crashes before recording success. Repeating the operation could append hello twice.
  - Technical: database commits and workspace writes are separate operations. One accepted command is not a promise of one model call or one file effect.
- Uncertain external work is marked interrupted rather than automatically resent.
  - Technical: a dispatch marker [saved record that an external attempt may have been sent] is written before sending to Codex. A crash before send can therefore also abandon an attempt that never ran.
  - Example: Stop does not undo edits or prove every child process has exited.

Show / demonstrate

1. If rehearsed on disposable data, show saved history after an idle service stop/start. Keep active crash tests separate.

If unavailable: Explain append → crash → unknown outcome using the table.

Limit to state: Native model calls may repeat under Durable recovery rules. Windows Bash shutdown and full Linux child cleanup remain unresolved checks.

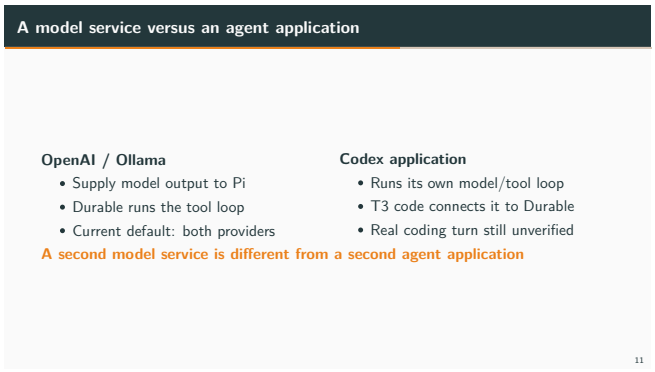
Transition: Codex has its own execution loop, which explains the special external boundary.

References: Wiki: llm wiki / 03 durability and recovery.md

Wiki: side effects checkpoints and dispatch markers

Wiki: llm wiki / interview questions / 04 failures and recovery.md

11. A model service versus an agent application



Target time: 35 seconds

Speaking points

- A provider [service supplying model output] is different from a harness [software running conversation and tools].
  - Example: OpenAI and Ollama supply model output to Durable. Durable runs their tool loop.
- An external harness [separate agent application] runs its own model/tool loop outside Durable.
  - Example: Codex runs as another process. Our adapter [connector translating requests/events] sends work and tracks its result; it is not just another model URL.
- We reused T3's actual Codex client/protocol code, not its entire app.
  - Technical: protocol [request/event format] tests use scripted child processes and recorded messages. Real CLI [command-line application] initialization works, but a real authenticated coding turn/continuation is still unverified.

Show / demonstrate

1. Point to the OpenAI/Ollama names, then explain Codex as a separate optional execution path.

If unavailable: Use the two execution paths; no live Codex demonstration is required without verified authentication.

Limit to state: Two native model providers do not prove two working agent applications. Other provider keys and real Codex authentication have separate checks.

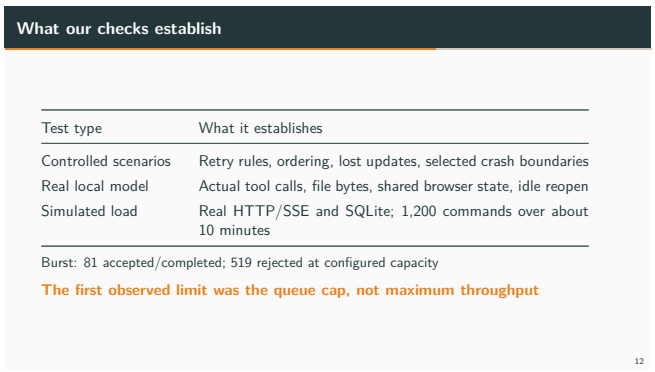
Transition: Here is the evidence we have for each claim.

References: Wiki: exactly what t3 contributed

Wiki: provider model agent harness and adapter

Source: packages / durable / src / adapters / codex / vendor / UPSTREAM.md

12. What our checks establish



Target time: 45 seconds

Speaking points

- Controlled tests force known timing; real-model checks establish actual integration.
  - Example: a test holds both reads before releasing either write. A separate real Ollama check verifies actual generated text, file bytes and reset.
- The load test exercised real HTTP and SQLite with simulated agent work.
  - Example: 1,200 commands completed in about ten minutes. A burst completed 81 and rejected 519 at the configured queue limit.
  - Technical: a soak [sustained workload] is not a real-model throughput benchmark. One observed start per accepted command is not a universal exactly-once guarantee.
- Every result needs its boundary stated.
  - Technical: 48 demo tests and 68 focused Durable tests passed in recorded runs; this is not the entire repository suite.
  - Example: local Ollama success does not prove GCP speed, real Codex behavior or correct handling of every crash.

Show / demonstrate

1. Show the saved load report or the portable Ollama proposal result. Do not run load on the presentation's live store.

If unavailable: Use the slide's counts and explain which work was simulated.

Limit to state: Maximum throughput, token overhead and cloud performance remain unmeasured; GCP slow-chat diagnosis is still open.

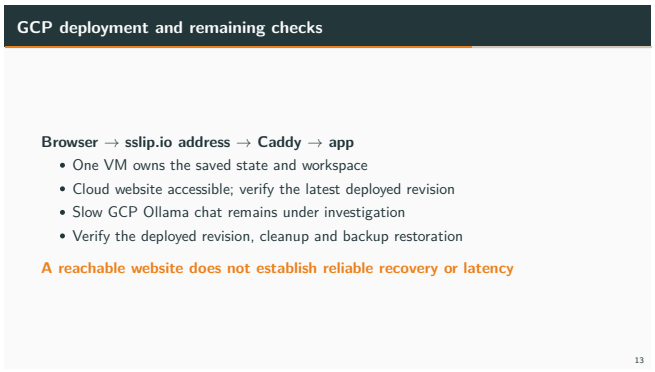
Transition: Deployment adds another set of operational checks.

References: Wiki: the load result worth remembering

Wiki: what the measurements cannot prove

Source: apps / collab demo / load / README.md

13. GCP deployment and remaining checks



Target time: 35 seconds

Speaking points

- The app runs on one VM [virtual machine] with saved data and a shared project folder.
  - Example: the sslip.io address routes to Caddy [server forwarding browser traffic securely to the app], which forwards to 127.0.0.1:4310.
- The cloud website is accessible, but model speed and recovery still need separate checks.
  - Example: regular Ollama chat on GCP can exceed 60 seconds while idle curl generation is fast. The cause has not been confirmed.
  - Technical: compare full model-request timing with browser delivery before attributing the delay to CPU, history or the proxy.
- A safe update rebuilds assets and restarts the single owner while preserving data and credentials.
  - Example: stop after agents are idle, back up, fetch the reviewed revision, build, start, then reload both pages.
  - Technical: a failure domain [things that fail together] includes the owning process, disk and VM/zone. Starting two owners on the same files is unsafe.

Show / demonstrate

1. Use only the actual tested URL/revision. Confirm service/model readiness before the interview; keep a local fallback ready.

If unavailable: Show the deployment path and explain the outstanding latency/restore checks.

Limit to state: Do not generalize website access into proven recovery or performance. Confirm the latest revision, backup restoration, Linux child cleanup and hosted demonstration timing.

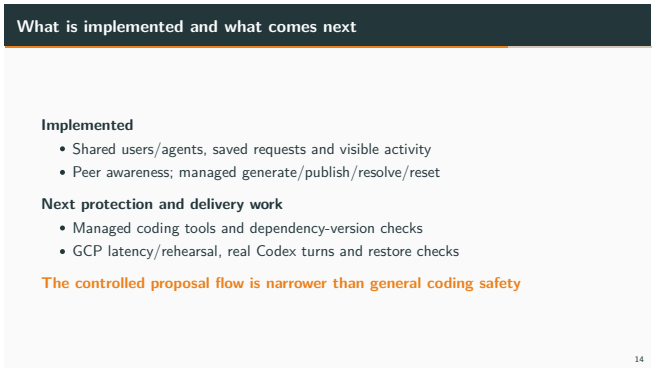
Transition: Finish with what we delivered and the next protection to add.

References: Wiki: why the proposed gcp deployment uses one vm

Wiki: llm wiki / interview questions / 08 scaling and operations.md

Source: apps / collab demo / deploy / README.md

14. What is implemented and what comes next



Target time: 40 seconds

Speaking points

- Implemented: shared agents, saved request identity, visible activity, peer awareness and the controlled managed-file flow.
  - Example: two users can continue the same agent conversation and demonstrate a refused stale proposal with explicit resolution/reset.
- Next, make ordinary coding edits participate in a managed protocol and track the files their plans depend on.
  - Example: if file2's API changes, refuse A's file1 proposal and request a new plan before publishing.
  - Technical: read set [files the plan relied on] validation needs coordination through publication. Prompt reminders alone cannot provide it.
- Keep further extensions separate from the current take-home scope.
  - Example: automatic re-drafting after resolution, worktrees [separate checked-out directories], shared approvals and multiple owners need additional design/tests.
  - Technical: current priorities include GCP latency/rehearsal, real Codex coding/continuation and Linux cleanup/backup restoration.

Show / demonstrate

1. Return to the shared-agent or managed conflict view for questions. Use backup slides only when relevant.

If unavailable: End with the concrete delivered behaviors and the dependency-change example.

Limit to state: Native rehearsal uses existing OpenAI/Ollama access; Codex needs separate authentication. Updated cloud code and acceptance checks are still required.

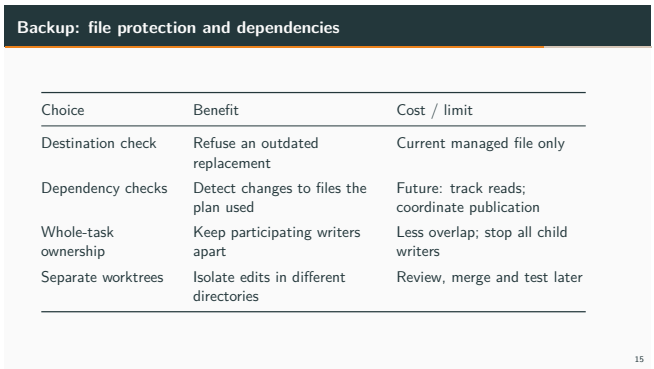
Transition: Invite questions about file protection, recovery or the reused components.

References: Wiki: what is tested and what to demonstrate next

Wiki: what would you implement next and what would you defer

Source: current risks and open checks

15. Backup: file protection and dependencies



Target time: Backup slide

Speaking points

- Today's managed check validates the destination file, not every file used to plan an edit.
  - Example: file1 is unchanged, but file2 now requires an object instead of a string. The destination check can pass while the code is wrong.
- Dependency-aware checks record versions of the files read and validate them before publication.
  - Technical: a read set [files the plan relied on] must remain unchanged through validation/publication, or another writer can race the check. All relevant writers must participate.
- Whole-task ownership or separate worktrees offer different trade-offs.
  - Example: ownership makes peers wait; worktrees allow isolated edits but require reviewed integration and tests.
  - Technical: a lease [permission that expires] cannot stop an old shell. T3's inspected startup lease ends before its provider turn, so it is not a whole-task lock.

Show / demonstrate

1. Use the file1/file2 example to compare destination checks, dependency checks, ownership and worktrees.

If unavailable: Walk through the rows without opening another tool.

Limit to state: Dependency validation, whole-task ownership, automatic merging and unrestricted shell protection are not implemented.

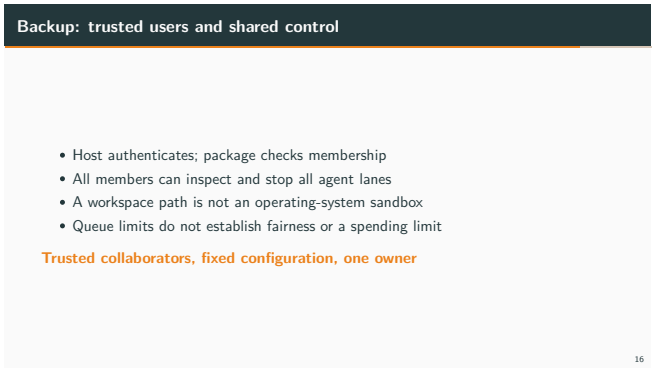
Transition: Return to the relevant main slide after answering.

References: Wiki: llm wiki / interview questions / 03 file conflicts.md

Wiki: locks ownership leases and fencing

Source: apps / collab demo / CONFLICT_RESOLUTION_PROPOSAL.md

16. Backup: trusted users and shared control



Target time: Backup slide

Speaking points

- The host identifies users, and the package checks their membership.
  - Example: a message saying ”I am Alice” does not authenticate its sender. A valid token maps to Alice before the package receives the command.
- Every current member can see all agent conversations and stop shared work.
  - Example: Bob’s Stop also affects Alice’s queued requests on that agent. There are no private per-user agent histories or dynamic roles.
- A workspace directory is not a sandbox [enforced restriction on what a process can access].
  - Example: ordinary coding-agent shell tools can access files available to the service account outside that directory.
  - Technical: queue limits bound admitted work, not money spent, hostile execution or fairness between users.

Show / demonstrate

1. Explain the trusted-team boundary; do not demonstrate secret access or destructive commands.

If unavailable: Use the token-versus-message example.

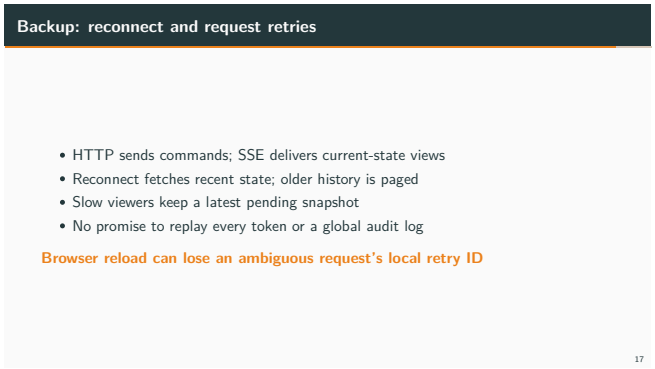
Limit to state: Public URL access still requires application authentication; hostile multi-tenant execution is outside this prototype.

Transition: Return to the shared-control or deployment discussion.

References: Wiki: authentication membership and authority

Wiki: llm wiki / interview questions / 06 security and api.md

17. Backup: reconnect and request retries



Target time: Backup slide

Speaking points

- Reconnect fetches the latest view; it does not replay every missed token or status change.
  - Technical: SSE [server-to-browser updates over HTTP] carries snapshots [recent views]. Saved message history can be loaded separately.
- A slow viewer keeps a recent pending view rather than every intermediate update.
  - Example: Bob may jump from running to completed after reconnect, without seeing each tool step.
  - Technical: coalescing [replacing an older pending view with the newest] limits that buffer, not all operating-system or process memory.
- The server's saved receipt and the browser's pending retry ID have different lifetimes.
  - Example: Retry same command reuses the ID; reload/relogin can lose it. Copy/paste with a new Send creates another command, even with identical text.
  - Technical: a lost acceptance reply does not prove rejection. Storage errors can occur after commit; stop the failed owner before reopening and retry the original identity.

Show / demonstrate

1. Reload only an idle/acknowledged browser view, sign in and show restored history without submitting another instruction.

If unavailable: Use the same-ID retry versus new-ID Send example.

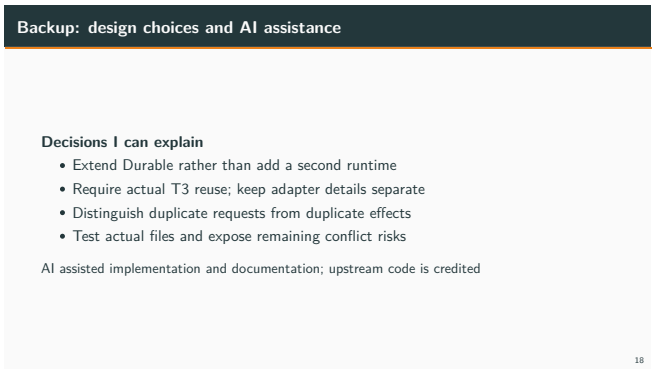
Limit to state: There is no complete event journal or persisted browser retry ledger. Discarding a retry does not cancel accepted work.

Transition: Return to the request-lifecycle slide.

References: Wiki: snapshots are convergent views not an event journal

Wiki: backpressure fan out and coalescing

18. Backup: design choices and AI assistance



Target time: Backup slide

Speaking points

- I directed the project toward extending Durable and demonstrating shared behavior with explicit limits.
  - Example: evolving the fork, requiring actual T3 reuse and challenging the original access-matrix demo were deliberate scope decisions.
- Existing Pi components and T3 code remain credited; AI assisted implementation and documentation.
  - Example: I should be able to explain the receipt transaction and stale-edit trace without relying on an assistant's wording.
- The original twenty-hour target shaped scope; it is not a measured elapsed-time claim.
  - Technical: one owner, fixed membership and one managed file reduce delivery complexity. Broader protection requires additional work and evidence.

Show / demonstrate

1. Answer with one design choice, one concrete trace and one test boundary.

If unavailable: Use the file conflict or lost-reply example.

Limit to state: Phrase personal ownership truthfully; do not imply all upstream code or validation was written manually.

Transition: Return to the question being discussed.

References: Wiki: how do you explain ai assisted development

Wiki: how would you explain your own judgment when much of the implementation used ai assistance